Introduction to Java

Java is a **high-level, object-oriented, class-based programming language** developed by Sun Microsystems. It was designed to be simple, secure, portable, and platform-independent.

Java programs are first converted into **bytecode** by the Java compiler. This bytecode can run on any system that has a **Java Virtual Machine (JVM)**.

Java is a high-level, object-oriented and platform-independent programming language used for developing a wide variety of applications.

Why is Java Popular?

Java became popular because of the following characteristics:

- Simple and easy to learn
- Object-oriented
- Platform-independent
- Secure
- Robust
- Portable
- Multithreaded
- Distributed
- High performance
- Supports network programming

History of Java

Java was developed at **Sun Microsystems** in the early 1990s.

The development team was led by **James Gosling**, along with other engineers including Mike Sheridan and Patrick Naughton.

Initially, the language was called **Oak**. It was designed for consumer electronic devices and embedded systems.

Later, the name was changed from Oak to **Java**.

Important Events in Java History

Year	Event
1991	Java project started at Sun Microsystems
1991	Language was initially called Oak
1995	Java was officially announced
1996	First major public Java development kit was released
2010	Oracle acquired Sun Microsystems
Present	Java continues to be widely used in software development

James Gosling is commonly known as the **father of Java** because he was the principal designer of the language.

Java and the Internet

Java became closely associated with the Internet because it allowed programs to run on different computers and operating systems.

Before Java, applications often had to be specifically developed or compiled for a particular operating system.

Java introduced the idea of:

Write Once, Run Anywhere (WORA)

This means that a Java program can be compiled into bytecode and then executed on different platforms using a suitable JVM.

Java's Role in Internet Programming

Java provides several features useful for network and Internet-based applications:

- Platform independence
- Network programming support
- Security
- Multithreading
- Distributed computing
- Support for web-based applications

Java can be used to develop:

- Web applications
- Server-side applications
- Network applications
- Enterprise applications
- Cloud-based software

Java Applets

A **Java Applet** was a small Java program designed to run inside a web browser or an applet viewer.

Applets were mainly used in the early days of Java for creating interactive content on web pages.

Characteristics of Applets

- They were small Java programs.
- They could be embedded in web pages.
- They were designed to execute in a controlled environment.
- They could provide interactive features.
- They used a restricted security model.

Basic Applet Life Cycle

An applet traditionally had several important methods:

1. `init()` – called when the applet is initialized.
2. `start()` – called when the applet starts or resumes.
3. `paint()` – used for displaying output.
4. `stop()` – called when the applet is stopped.
5. `destroy()` – called before the applet is removed.

Important Note

Java applets are now **obsolete**. Modern web browsers no longer support the Java browser-plugin technology required for traditional applets.

Therefore, applets are mainly studied today from a **historical and academic perspective**.

Java Applications

A Java application is a standalone program written in Java that runs using the Java platform.

Unlike traditional applets, Java applications do not require a web browser.

Examples of Java Applications

Java is used to develop:

- Desktop applications
- Web applications
- Enterprise applications
- Banking software
- E-commerce systems
- Mobile applications, historically and through various Java-based platforms
- Network applications
- Server applications
- Big-data and distributed systems

Simple Java Application

```
class Hello
{
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
}
```

Explanation

- `class Hello` → Defines a class named `Hello`.
- `main()` → Starting point of a traditional Java application.
- `System.out.println()` → Displays output on the screen.

Features of Java

Java has several important features that make it different from many other programming languages.

1. Simple

Java was designed to be easier to learn and use than languages such as C++.

It removes or simplifies several complex features, such as explicit pointer arithmetic and manual memory management.

2. Object-Oriented

Java follows the object-oriented programming approach.

The major concepts of OOP include:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

These concepts help in developing modular and reusable software.

3. Platform Independent

Java is platform-independent because Java source code is compiled into **bytecode**.

The bytecode is executed by the JVM.

Process

```
Java Source Code
       ↓
    Compiler
       ↓
   Bytecode
       ↓
      JVM
       ↓
Operating System
```

Therefore, the same bytecode can generally run on different operating systems that provide a compatible Java runtime.

4. Portable

Java programs are portable because Java's primitive data types have defined sizes and Java bytecode is designed to run on different platforms through the JVM.

5. Secure

Java provides several security mechanisms, including:

- No direct pointer arithmetic
- Bytecode verification
- Class loading mechanisms
- Runtime security checks
- Controlled access to system resources

6. Robust

Java is considered robust because it provides:

- Strong type checking
- Exception handling
- Automatic memory management
- Garbage collection

These features help reduce many common programming errors.

7. Multithreaded

Java supports **multithreading**.

A thread is a lightweight unit of execution.

Multithreading allows a program to perform multiple tasks concurrently.

For example, a program may:

- Download data
- Process data
- Respond to user input

at the same time.

8. Distributed

Java provides libraries and technologies for developing distributed and network-based applications.

It supports communication between computers over networks.

9. High Performance

Java uses **Just-In-Time (JIT) compilation** in modern JVM implementations.

JIT compilation can convert frequently executed bytecode into native machine code at runtime, improving execution performance.

10. Dynamic

Java supports dynamic class loading and runtime linking.

Classes can be loaded when they are needed during program execution.

Basics of Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming approach in which a program is designed around **objects** and the classes that define them.

An object represents an entity that has:

- State
- Behaviour
- Identity

Example

Consider a `Student`.

Its state may include:

```
Name
Roll Number
Course
Age
```

Its behaviour may include:

```
Study()
AttendClass()
GiveExam()
```

Class

A **class** is a blueprint or template used to create objects.

It defines the data and behaviour that objects of that class can have.

Object

An **object** is an instance of a class.

For example:

```
Student s1 = new Student();
```

Here:

- `Student` → Class
- `s1` → Reference variable
- `new Student()` → Creates an object

Real-Life Example

```
Class  → Car
Object → BMW car
Object → Audi car
Object → Toyota car
```

The class provides the general structure, while individual objects represent particular instances.

Encapsulation

Encapsulation means wrapping data and methods together inside a class and controlling access to the data.

Example:

```
class Student
{
    private int marks;

    public void setMarks(int m)
    {
        marks = m;
    }

    public int getMarks()
    {
        return marks;
```

```
    }
}
```

Here, `marks` is private and can be accessed through methods.

Advantages

- Data security
- Better control over data
- Improved maintainability
- Prevents unwanted direct access

Inheritance

Inheritance is the mechanism by which one class acquires properties and behaviours of another class.

The existing class is called the **superclass/parent class**, and the new class is called the **subclass/child class**.

Example:

```
class Animal
{
    void eat()
    {
        System.out.println("Eating");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking");
    }
}
```

Here, `Dog` inherits the `eat()` method from `Animal`.

Advantages

- Code reusability
- Reduces duplication
- Supports hierarchical classification
- Makes programs easier to maintain

Polymorphism

The word polymorphism means **"many forms."**

Polymorphism allows the same interface or method name to represent different behaviours.

Two important forms are:

1. Compile-Time Polymorphism

Usually achieved through **method overloading**.

```
class Calculator
{
    int add(int a, int b)
    {
        return a + b;
    }

    int add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

2. Runtime Polymorphism

Usually achieved through **method overriding**.

```
class Animal
{
    void sound()
    {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal
{
    void sound()
    {
        System.out.println("Bark");
    }
}
```

Abstraction

Abstraction means hiding unnecessary implementation details and showing only the essential features.

For example, when we use an ATM, we know how to:

- Insert a card
- Enter a PIN
- Withdraw money

but we do not need to know the internal implementation of the banking system.

Java supports abstraction using:

- Abstract classes
- Interfaces

Main Principles of OOP

The four major principles of object-oriented programming are:

Principle	Meaning
Encapsulation	Binding data and methods together and controlling access
Inheritance	Acquiring properties and behaviour from another class
Polymorphism	One interface/method having different forms
Abstraction	Hiding implementation details and showing essential features

These are commonly called the **four pillars of OOP**.

How Java is Different from C

Java and C are both programming languages, but their programming approaches are different.

Basis	C	Java
Programming approach	Procedural	Object-oriented
Platform	Generally platform-dependent binaries	Platform-independent through JVM
Pointers	Supports pointers and pointer arithmetic	No explicit pointer arithmetic
Memory management	Manual	Automatic garbage collection
Preprocessor	Supports preprocessor directives	No C-style preprocessor
Multiple inheritance	Not applicable through classes	Not supported through classes; interfaces are used
Exception handling	No built-in exception handling mechanism like Java	Built-in exception handling
Security	Relatively lower-level	Stronger runtime security features
Compilation	Compiles to native machine code	Compiles to bytecode
Main program	`main()` is the program entry point	`main()` is the traditional entry point for standalone applications

How Java is Different from C++

Java was influenced by C++, but it intentionally avoids or changes several C++ features.

Basis	C++	Java
Platform	Generally platform-dependent binaries	Platform-independent through JVM
Programming approach	Multi-paradigm, including OOP	Primarily class-based OOP
Pointers	Supports pointers	No explicit pointer arithmetic
Memory management	Manual/RAII mechanisms	Automatic garbage collection
Multiple inheritance of classes	Supported	Not supported for classes
Operator overloading	Supported	Generally not supported for user-defined classes
Preprocessor	Supports	No C++-style preprocessor
Header files	Used	Not used in the C++ sense
Exception handling	Supported	Supported
Templates	Supported	Generics are used instead
Virtual functions	Explicitly used for runtime polymorphism	Methods are dynamically dispatched where applicable
Global functions	Supported	Methods belong to classes
Friend functions/classes	Supported	No equivalent `friend` mechanism
Portability	Depends on compilation environment	Bytecode + JVM provides strong portability

Java Architecture: JDK, JRE and JVM

Three important terms associated with Java are:

JVM – Java Virtual Machine

JVM is the virtual machine that executes Java bytecode.

Its major responsibilities include:

- Loading classes
- Verifying bytecode
- Executing bytecode
- Managing memory
- Providing runtime services

JRE – Java Runtime Environment

JRE provides the environment required to **run** Java applications.

Conceptually:

```
JRE = JVM + Required Runtime Libraries
```

JDK – Java Development Kit

JDK provides tools required to **develop** Java programs.

Conceptually:

```
JDK = Development Tools + JRE-related Runtime Components
```

The exact packaging of JDK/JRE has changed in modern Java releases, but the conceptual distinction remains useful for learning.

Java Compilation and Execution

A Java program generally follows these steps:

Step 1: Write Source Code

Example:

```
class Test
{
    public static void main(String args[])
    {
        System.out.println("Hello");
    }
}
```

Save the file as:

```
Test.java
```

Step 2: Compile

The Java compiler converts source code into bytecode.

```
javac Test.java
```

This produces:

```
Test.class
```

Step 3: Execute

The JVM executes the bytecode:

```
java Test
```

Complete Flow

```
Test.java
   ↓
Java Compiler (javac)
   ↓
Test.class
   ↓
Bytecode
   ↓
JVM
   ↓
Machine/OS
```

Advantages of Java

Major advantages of Java include:

1. Platform independence
2. Object-oriented programming
3. Automatic memory management
4. Security features
5. Robustness
6. Multithreading support
7. Network programming support
8. Large standard library
9. Portability
10. Strong ecosystem and tooling

Limitations of Java

Some limitations of Java include:

- Generally more memory consumption than low-level languages such as C.
- Startup and runtime overhead can exist because of the JVM.
- It provides less direct control over hardware than C/C++.
- Garbage collection introduces runtime overhead.
- Syntax can be more verbose than some modern languages.

Program Structure in Java

A Java program is written using **classes, methods, statements, variables, and other program elements**.

A simple Java program has the following basic structure:

```
class Student
{
    public static void main(String args[])
    {
        System.out.println("Hello Java");
    }
}
```

1. Class Declaration

```
class Student
```

It defines a class named `Student`.

A Java program generally contains one or more classes.

2. Main Method

```
public static void main(String args[])
```

The `main()` method is the traditional entry point for a standalone Java application.

3. Statement

```
System.out.println("Hello Java");
```

This statement displays output on the screen.

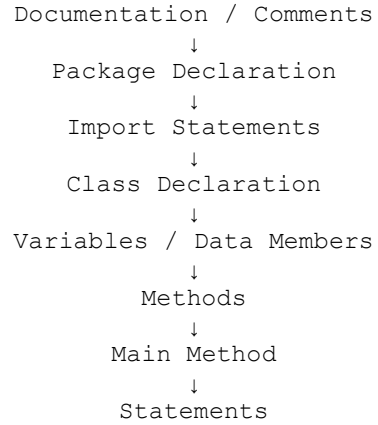
4. Curly Braces

```
{
    ...
}
```

Curly braces define a block of Java code.

General Structure of a Java Program

A Java source file may contain the following components:



Example:

```
// Simple Java Program

import java.util.*;
```

```
class Demo
{
    int number = 10;

    void display()
    {
        System.out.println(number);
    }

    public static void main(String args[])
    {
        Demo d = new Demo();
        d.display();
    }
}
```

Important Point

Not every Java program must contain all these sections. For example, a small program may not need an `import` statement or a package declaration.

Simple Java Programs

Program 1: Print Hello World

```
class Hello
{
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
}
```

Output

```
Hello World
```

Program 2: Addition of Two Numbers

```
class Addition
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 20;
        int sum = a + b;

        System.out.println("Sum = " + sum);
    }
}
```

Output

```
Sum = 30
```

Java Tokens

A **token** is the smallest individual unit of a Java program that has a meaningful role in the language.

For example:

```
int age = 20;
```

The tokens are:

```
int
age
=
20
;
```

Java tokens are broadly classified into:

1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Separators

1. Keywords

Keywords are reserved words that have a predefined meaning in Java.

They cannot normally be used as identifiers such as variable or class names.

Examples

```
Class, public, static
Void, int, If, else, for, while
Return, new
Extends, implements
Private, protected
```

Example:

```
class Student
{
    int age;
}
```

Here:

- `class` → Keyword
- `int` → Keyword
- `Student` → Identifier
- `age` → Identifier

2. Identifiers

An **identifier** is the name given to a program element such as:

- Class
- Method
- Variable
- Interface
- Object/reference variable

Example:

```
class Student
{
    int marks;

    void display()
    {
        System.out.println(marks);
    }
}
```

Here:

- `Student` → Identifier
- `marks` → Identifier
- `display` → Identifier

Rules for Naming Identifiers

The following rules should be followed:

1. An identifier can contain letters, digits, `_`, and `$`.
2. It must not start with a digit.
3. It cannot be a Java keyword.
4. Java identifiers are **case-sensitive**.
5. Spaces are not allowed.
6. Special characters such as `@`, `#`, `%`, etc. are not allowed.

Valid Identifiers

```
age
studentName
marks1
_total
$amount
```

Invalid Identifiers

```
1student       // Starts with digit
student name   // Contains space
class          // Keyword
student-name   // Hyphen not allowed
```

3. Literals

A **literal** is a fixed value directly written in a Java program.

Examples:

```
int age = 20;
double price = 99.50;
char grade = 'A';
boolean result = true;
String name = "Rahul";
```

Here:

- `20` → Integer literal
- `99.50` → Floating-point literal

- `'A'` → Character literal
- `true` → Boolean literal
- `"Rahul"` → String literal

Types of Literals

Common types include:

1. Integer literals
2. Floating-point literals
3. Character literals
4. String literals
5. Boolean literals
6. `null` literal

4. Operators

Operators are symbols used to perform operations on values or variables.

Arithmetic Operators

```
+   Addition
-   Subtraction
*   Multiplication
/   Division
%   Remainder
```

Example:

```
int c = a + b;
```

Relational Operators

```
==   Equal to
!=   Not equal to
>    Greater than
<    Less than
>=   Greater than or equal to
<=   Less than or equal to
```

Logical Operators

```
&&   Logical AND
||   Logical OR
!    Logical NOT
```

Assignment Operators

```
= , +=, -=, *=, /=, %=
```

5. Separators

Separators are symbols that help define the structure of Java programs.

Common separators include:

```
( )     Parentheses
{ }     Braces
[ ]     Brackets
```

```
;       Semicolon
,       Comma
.       Dot
```

Example:

```
System.out.println("Hello");
```

Here:

- `.` separates `System` and `out`
- `(` and `)` enclose method arguments
- `;` terminates the statement

Java Statements

A **statement** is a complete instruction that tells the Java program to perform an operation.

Example:

```
int x = 10;
```

This is a declaration and initialization statement.

Another example:

```
System.out.println(x);
```

This is an output statement.

Types of Java Statements

Java statements can be broadly classified as follows:

1. Declaration Statements

Used to declare variables.

```
int age;
double salary;
```

2. Expression Statements

Used to perform an operation.

```
x = 10;
x++;
sum = a + b;
```

3. Selection Statements

Used to make decisions.

```
if
if-else
switch
```

Example:

```
if(age >= 18)
{
    System.out.println("Eligible");
}
```

4. Iteration Statements

Used for repetition.

```
for
while
do-while
```

Example:

```
for(int i = 1; i <= 5; i++)
{
    System.out.println(i);
}
```

5. Jump Statements

Used to transfer control.

```
break
continue
return
```

Java Virtual Machine (JVM)

JVM stands for Java Virtual Machine.

It is a virtual machine that executes Java bytecode.

The JVM is one of the most important components of the Java platform.

Working of JVM

```
Java Source Code
       ↓
    javac
       ↓
    Bytecode
       ↓
      JVM
       ↓
Machine Code / Execution
```

Example

Suppose we have:

```
Hello.java
```

After compilation:

```
Hello.class
```

The `.class` file contains Java bytecode.

The JVM loads and executes this bytecode.

Main Functions of JVM

The JVM performs several important tasks:

1. Class Loading

The JVM loads required class files into memory.

2. Bytecode Verification

It checks bytecode for certain structural and security constraints before execution.

3. Execution

It executes bytecode using interpretation and/or JIT compilation, depending on the JVM implementation.

4. Memory Management

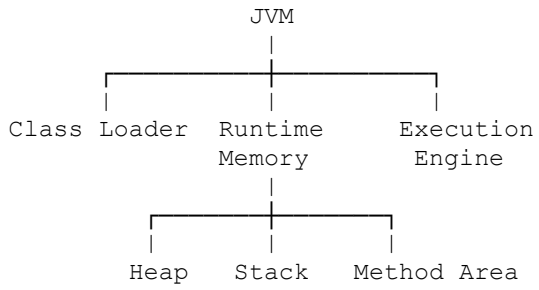
The JVM manages memory used by Java programs.

5. Garbage Collection

The JVM automatically identifies and reclaims memory occupied by objects that are no longer reachable.

JVM Components

A simplified view of JVM components is:



Class Loader

Loads classes into JVM memory.

Runtime Data Areas

Important runtime memory areas include:

- Heap
- JVM stacks
- Method area / class-related runtime data

Execution Engine

Executes the bytecode.

It may use:

- Interpreter
- JIT compiler

Command Line Arguments

Command line arguments are values passed to a Java program when it is started from the command line.

They are received through the `String[] args` parameter of the `main()` method.

Syntax

```
public static void main(String args[])
```

or

```
public static void main(String[] args)
```

Both forms are valid.

Example

```
class Demo
{
    public static void main(String args[])
    {
        System.out.println(args[0]);
    }
}
```

Suppose we run:

```
java Demo Rahul
```

Output

```
Rahul
```

Here:

```
args[0] = "Rahul"
```

Example with Multiple Arguments

```
class Demo
{
    public static void main(String args[])
    {
        System.out.println("Name: " + args[0]);
        System.out.println("Course: " + args[1]);
    }
}
```

Run:

```
java Demo Rahul BCA
```

Output:

```
Name: Rahul
Course: BCA
```

Important Point

Command line arguments are received as **strings**.

If you want to use a number as an integer, it must be converted.

Example:

```
int n = Integer.parseInt(args[0]);
```

Constants in Java

A **constant** is a value that should not be changed after it has been assigned.

Java uses the keyword **`final`** to declare a variable whose value cannot be reassigned.

Syntax

```
final dataType CONSTANT_NAME = value;
```

Example:

```
final int MAX = 100;
```

Once initialized, the variable cannot be assigned another value.

```
final int MAX = 100;

// MAX = 200;   // Error
```

Example Program

```
class ConstantDemo
{
    public static void main(String args[])
    {
        final double PI = 3.14159;

        System.out.println("PI = " + PI);
    }
}
```

Naming Convention for Constants

Constants are commonly written in:

UPPERCASE_WITH_UNDERSCORES

Examples:

```
MAX_VALUE
MIN_VALUE
```

```
PI
COLLEGE_NAME
```

Variables in Java

A **variable** is a named memory location/reference used to hold a value that may change during program execution.

Example:

```
int age = 20;
```

Here:

- `int` → Data type
- `age` → Variable name
- `20` → Initial value

Example:

```
age = 21;
```

The value of `age` can be changed.

Variable Naming Conventions

Java follows commonly accepted naming conventions.

Rules

Variable names should generally:

- Start with a lowercase letter.
- Use camelCase for multiple words.
- Be meaningful and descriptive.
- Avoid unnecessary abbreviations.

Examples

```
studentName
totalMarks
accountBalance
studentAge
```

Avoid

```
x
abc
xyz1
```

unless the short name is appropriate for a small/local context.

Class Naming Convention

Class names should generally:

- Start with an uppercase letter.
- Use **PascalCase** when multiple words are used.
- Be nouns or noun phrases where appropriate.
- Clearly describe the purpose of the class.

Examples

```
Student
Employee
BankAccount
StudentRecord
OnlinePayment
```

Example

```
class StudentRecord
{
    // class body
}
```

Here, `StudentRecord` follows the standard Java class naming convention.

Method Naming Convention

Methods generally use:

lowerCamelCase

Examples:

```
calculateSalary()
displayStudent()
getMarks()
setName()
calculateTotal()
```

Example:

```
void displayStudent()
{
    System.out.println("Student Details");
}
```

Package Naming Convention

Package names are generally written in **lowercase**.

Examples:

```
java.util
java.io
com.company.project
```

For custom packages, organizations often use reverse-domain naming.

Example:

```
com.example.student
```

Complete Naming Convention Table

Program Element	Convention	Example
Class	PascalCase	`StudentRecord`
Interface	PascalCase	`RunnableTask`
Method	lowerCamelCase	`calculateTotal()`
Variable	lowerCamelCase	`studentName`
Constant	UPPER_CASE	`MAX_VALUE`
Package	lowercase	`com.example.project`

Comments in Java

Comments are notes written in source code for programmers. They are ignored by the compiler.

Single-Line Comment

```
// This is a comment
```

Multi-Line Comment

```
/*
   This is a
   multi-line comment
*/
```

Documentation Comment

```
/**
 * This method displays student details.
 */
```

Documentation comments can be processed by tools such as Javadoc.

Data Types in Java

A **data type** specifies what type of value a variable can store and what kind of operations can be performed on that value.

Example:

```
int age = 20;
```

Here:

- `int` → Data type
- `age` → Variable
- `20` → Value

Java data types are mainly divided into two categories:

```
Data Types
     │
     ├── Primitive Data Types
     │
     └── Non-Primitive / Reference Data Types
```

Primitive Data Types

Primitive data types are the basic data types provided by Java.

Java has **8 primitive data types**:

Data Type	Typical Size	Example
`byte`	8 bits	`byte age = 20;`
`short`	16 bits	`short marks = 500;`
`int`	32 bits	`int salary = 50000;`
`long`	64 bits	`long population = 1000000L;`
`float`	32 bits	`float price = 25.5f;`
`double`	64 bits	`double pi = 3.14159;`
`char`	16 bits	`char grade = 'A';`
`boolean`	JVM-dependent representation	`boolean pass = true;`

1. byte

The `byte` data type stores small integer values.

```
byte age = 20;
```

Range:

```
-128 to 127
```

2. short

The `short` data type stores a larger integer range than `byte`.

```
short marks = 500;
```

Range:

```
-32,768 to 32,767
```

3. int

`int` is commonly used for whole-number values.

```
int salary = 50000;
```

Range:

```
-2^31 to 2^31 - 1
```

4. long

`long` is used when an integer value is larger than the normal `int` range.

```
long population = 1400000000L;
```

The suffix `L` is commonly used to indicate a `long` literal.

5. float

`float` is used for single-precision floating-point values.

```
float price = 99.50f;
```

The suffix `f` is required for a floating-point literal assigned directly to a `float`.

6. double

`double` is used for double-precision floating-point values.

```
double salary = 25000.75;
```

A decimal literal is `double` by default.

7. char

`char` stores a single UTF-16 code unit.

```
char grade = 'A';
```

A character is written inside **single quotation marks**.

Example:

```
char ch = 'B';
```

8. boolean

`boolean` stores either:

```
true
false
```

Example:

```
boolean result = true;
```

Non-Primitive / Reference Data Types

Reference types are used to refer to objects.

Examples include:

- Classes
- Arrays
- Interfaces
- Strings
- Enumerations

Example:

```
String name = "Rahul";
```

Here, `String` is a reference type.

Example of Array

```
int marks[] = {80, 75, 90};
```

Primitive vs Reference Data Types

Primitive	Reference
Basic data types	Refer to objects/data structures
Examples: `int`, `char`, `double`	Examples: `String`, arrays, classes
Hold values directly	Variables generally hold references
Have predefined types	Can be user-defined through classes/interfaces

Type Casting in Java

Type casting means converting a value from one data type to another.

For example:

```
int x = 10;
double y = x;
```

Here, the `int` value is converted to `double`.

There are two major forms:

1. Widening conversion
2. Narrowing conversion

1. Widening Type Casting

Widening conversion means converting a smaller compatible numeric type into a larger compatible numeric type.

It is generally performed automatically.

Example:

```
int x = 100;
double y = x;
```

Here:

```
int → double
```

No explicit cast is required.

Common Widening Order

```
byte
  ↓
short
  ↓
int
  ↓
long
  ↓
float
  ↓
double
```

`char` can also participate in numeric conversions.

Example:

```
int x = 65;
double y = x;
```

2. Narrowing Type Casting

Narrowing conversion means converting a larger numeric type into a smaller compatible numeric type.

It generally requires an explicit cast.

Syntax

```
dataType variable = (dataType) value;
```

Example:

```
double x = 25.75;
int y = (int)x;
```

Output value:

```
25
```

The fractional part is discarded.

Type Conversion vs Type Casting

Type Conversion

A conversion may happen automatically when Java can safely widen a value.

```
int a = 10;
double b = a;
```

Type Casting

The programmer explicitly specifies the target type.

```
double a = 10.5;
int b = (int)a;
```

Operators in Java

An **operator** is a symbol that performs an operation on one or more operands.

Example:

```
int c = a + b;
```

Here:

- `a` and `b` → Operands

- `+` → Operator
- `a + b` → Expression

Types of Operators

Java provides several categories of operators:

1. Arithmetic operators
2. Unary operators
3. Relational operators
4. Logical operators
5. Assignment operators
6. Conditional/Ternary operator
7. Bitwise operators
8. Shift operators
9. `instanceof` operator

1. Arithmetic Operators

Arithmetic operators perform mathematical operations.

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Remainder

Example:

```
int a = 20;
int b = 6;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Output:

```
26
14
120
3
2
```

2. Unary Operators

Unary operators operate on one operand.

Important unary operators include:

```
+    Unary plus
-    Unary minus
++   Increment
--   Decrement
!    Logical NOT
~    Bitwise complement
```

Example:

```
int x = 10;

x++;
```

Now:

```
x = 11
```

Increment Operator

The `++` operator increases a value by 1.

Post-Increment

```
int x = 10;
int y = x++;
```

First, `y` gets `10`, then `x` becomes `11`.

Pre-Increment

```
int x = 10;
int y = ++x;
```

First, `x` becomes `11`, then `y` gets `11`.

Decrement Operator

The `--` operator decreases a value by 1.

Post-Decrement

```
int x = 10;
int y = x--;
```

`y` gets `10`, then `x` becomes `9`.

Pre-Decrement

```
int x = 10;
int y = --x;
```

`x` becomes `9`, then `y` gets `9`.

3. Relational Operators

Relational operators compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
int a = 20;
int b = 10;

System.out.println(a > b);
```

Output:

```
true
```

Relational expressions generally produce a `boolean` result.

4. Logical Operators

Logical operators are mainly used with boolean expressions.

Operator	Meaning
`&&`	Logical AND
`||`	Logical OR
`!`	Logical NOT

Logical AND

```
if(age >= 18 && citizen == true)
```

Both conditions must be true.

Logical OR

```
if(day == 6 || day == 7)
```

At least one condition must be true.

Logical NOT

```
if(!flag)
```

It reverses the boolean value.

5. Assignment Operators

Assignment operators assign values to variables.

Operator	Example	Meaning
`=`	`x = 10`	Assign
`+=`	`x += 5`	`x = x + 5`
`-=`	`x -= 5`	`x = x - 5`
`*=`	`x *= 5`	`x = x * 5`
`/=`	`x /= 5`	`x = x / 5`
`%=`	`x %= 5`	`x = x % 5`

6. Conditional / Ternary Operator

The ternary operator is a short form of a simple `if-else` expression.

Syntax

```
condition ? expression1 : expression2;
```

Example:

```
int age = 20;

String result = (age >= 18) ? "Adult" : "Minor";

System.out.println(result);
```

Output:

```
Adult
```

7. Bitwise Operators

Bitwise operators work on integer types at the bit level.

Operator	Meaning
&	Bitwise AND
`	` Bitwise OR
^	Bitwise XOR
~	Bitwise complement

Example:

```
int a = 5;
int b = 3;

System.out.println(a & b);
```

8. Shift Operators

Shift operators move bits left or right.

Operator	Meaning
<<	Left shift
>>	Signed right shift
>>>	Unsigned right shift

Example:

```
int x = 8;

System.out.println(x << 1);
```

Output:

```
16
```

9. instanceof Operator

The `instanceof` operator checks whether an object is an instance of a specified type.

Example:

```
String name = "Java";

System.out.println(name instanceof String);
```

Output:

```
true
```

Expressions in Java

An **expression** is a combination of variables, literals, operators, method calls, and other elements that produces a value.

Example:

```
a + b
```

Another example:

```
x * y + 10
```

Example:

```
int result = a + b * 2;
```

Here:

```
a + b * 2
```

is an expression.

Types of Expressions

Common expressions include:

Arithmetic Expression

```
a + b * c
```

Relational Expression

```
a > b
```

Logical Expression

```
a > 10 && b < 20
```

Assignment Expression

```
x = 50
```

Conditional Expression

```
age >= 18 ? "Adult" : "Minor"
```

Operator Precedence

When an expression contains multiple operators, Java follows operator precedence and associativity rules.

Example:

```
int result = 10 + 5 * 2;
```

Multiplication is performed before addition.

Therefore:

```
5 * 2 = 10
10 + 10 = 20
```

So:

```
result = 20
```

Use Parentheses for Clarity

```
int result = (10 + 5) * 2;
```

Now:

```
15 * 2 = 30
```

Operator Type	Category	Precedence	Associativity
Unary	postfix	a++, a--	Right to left
	prefix	++a, --a, +a, -a, ~, !	Right to left
Arithmetic	Multiplication	*, /, %	Left to Right
	Addition	+, -	Left to Right
Shift	Shift	<<, >>, >>>	Left to Right
Relational	Comparison	<, >, <=, >=, instanceOf	Left to Right
	equality	==, !=	Left to Right
Bitwise	Bitwise AND	&	Left to Right
	Bitwise exclusive OR	^	Left to Right
	Bitwise inclusive OR	|	Left to Right
Logical	Logical AND	&&	Left to Right
	Logical OR	||	Left to Right
Ternary	Ternary	? :	Right to Left
Assignment	assignment	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=	Right to Left

Decision Making and Branching

Decision-making statements allow a program to choose different paths based on conditions.

For example:

```
If condition is true
        ↓
   Do this
Else
        ↓
   Do that
```

Java provides:

1. `if`
2. `if-else`
3. `else-if` ladder
4. Nested `if`
5. `switch`
6. Conditional/ternary operator

1. if Statement

The `if` statement executes a block when a condition is true.

Syntax

```
if(condition)
{
    // statements
}
```

Example

```
int age = 20;

if(age >= 18)
{
    System.out.println("Eligible");
}
```

Output:

```
Eligible
```

2. if-else Statement

The `if-else` statement provides two alternatives.

Syntax

```
if(condition)
{
    // statements
}
else
{
    // statements
}
```

Example

```
int number = 7;

if(number % 2 == 0)
{
    System.out.println("Even");
}
else
{
    System.out.println("Odd");
}
```

Output:

```
Odd
```

3. else-if Ladder

The `else-if` ladder is used when there are multiple conditions.

Example

```
int marks = 75;

if(marks >= 90)
{
    System.out.println("Grade A+");
}
else if(marks >= 75)
{
    System.out.println("Grade A");
}
else if(marks >= 60)
{
    System.out.println("Grade B");
}
else if(marks >= 50)
{
    System.out.println("Grade C");
}
else
{
    System.out.println("Fail");
}
```

Output:

```
Grade A
```

4. Nested if

An `if` statement placed inside another `if` statement is called a **nested if**.

Example:

```
int age = 20;
boolean citizen = true;

if(age >= 18)
{
    if(citizen)
    {
        System.out.println("Eligible");
    }
}
```

5. switch Statement

The `switch` statement is used when one expression needs to be compared against multiple possible values.

Syntax

```
switch(expression)
{
    case value1:
        // statements
        break;

    case value2:
        // statements
        break;

    default:
        // statements
}
```

Example

```
int day = 2;

switch(day)
{
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Invalid day");
}
```

Output:

```
Tuesday
```

6. Importance of break in Switch

The `break` statement normally terminates the current `switch` case and exits the switch.

Example:

```
switch(day)
{
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;
}
```

If `break` is omitted, execution can continue into subsequent cases. This is called **fall-through**.

7. Default Case

The `default` case executes when none of the specified cases matches.

Example:

```
int choice = 5;

switch(choice)
{
    case 1:
        System.out.println("Add");
        break;

    case 2:
        System.out.println("Subtract");
        break;

    default:
        System.out.println("Invalid choice");
}
```

Output:

```
Invalid choice
```

Ternary Operator as Decision Making

The conditional operator can be used for simple decisions.

Example:

```
int a = 10;
int b = 20;

int max = (a > b) ? a : b;

System.out.println(max);
```

Output:

```
20
```

It is useful for short expressions, but complex decisions are generally clearer with `if-else`.

Branching Statements

Branching or jump statements change the normal flow of program execution.

Important branching statements are:

1. `break`
2. `continue`
3. `return`

`break` Statement

The `break` statement terminates the nearest loop or `switch` statement.

Example:

```
for(int i = 1; i <= 10; i++)
{
    if(i == 5)
    {
        break;
    }

    System.out.println(i);
}
```

Output:

```
1
2
3
4
```

`continue` Statement

The `continue` statement skips the remaining statements of the current loop iteration and proceeds to the next iteration.

Example:

```
for(int i = 1; i <= 5; i++)
{
    if(i == 3)
    {
        continue;
    }
    System.out.println(i);
}
```

Output:

```
1
2
4
5
```

`return` Statement

The `return` statement exits from a method and may return a value.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here, `return` sends the calculated value back to the caller.

For a `void` method:

```
void display()
{
```

```
    System.out.println("Hello");
    return;
}
```

The `return` statement simply exits the method.

Difference Between `break`, `continue` and `return`

Statement	Purpose
`break`	Terminates the nearest loop or switch
`continue`	Skips current loop iteration
`return`	Exits the current method

Complete Example: Decision Making

```
class Result
{
    public static void main(String[] args)
    {
        int marks = 82;
        if(marks >= 90)
        {
            System.out.println("Grade A+");
        }
        else if(marks >= 75)
        {
            System.out.println("Grade A");
        }
        else if(marks >= 60)
        {
            System.out.println("Grade B");
        }
        else if(marks >= 50)
        {
            System.out.println("Grade C");
        }
        else
        {
            System.out.println("Fail");
        }
    }
}
```

Output:

```
Grade A
```